



BOYZ TRADE

Product Requirements Document — MVP v1.0

Virtual Crypto Trading Platform

Confidential — Draft Ready for Development

Document Version	1.0 — MVP
Product Name	Boyz Trade
Status	Draft — Ready for Development
Target Platforms	Web (Next.js) + Mobile (Flutter)
Target Users	Beginners who want to learn crypto trading safely
Goal	Launch a working MVP that proves the concept and serves as a portfolio piece

01 Product Overview

What we are building and why

1.1 What is Boyz Trade?

Boyz Trade is a virtual crypto trading simulator. Think of it like a flight simulator for pilots — before a pilot flies a real plane, they practice in a simulator with zero real risk. Boyz Trade is the same idea but for trading crypto. Users get fake money (virtual balance) and trade real market prices — with zero real money at risk.

1.2 Problem We Are Solving

- Most beginners are afraid to trade real crypto because they might lose real money.
- There is no safe place to practice and understand how buying/selling crypto actually works.
- Boyz Trade gives them a risk-free environment to learn, experiment, and track results.

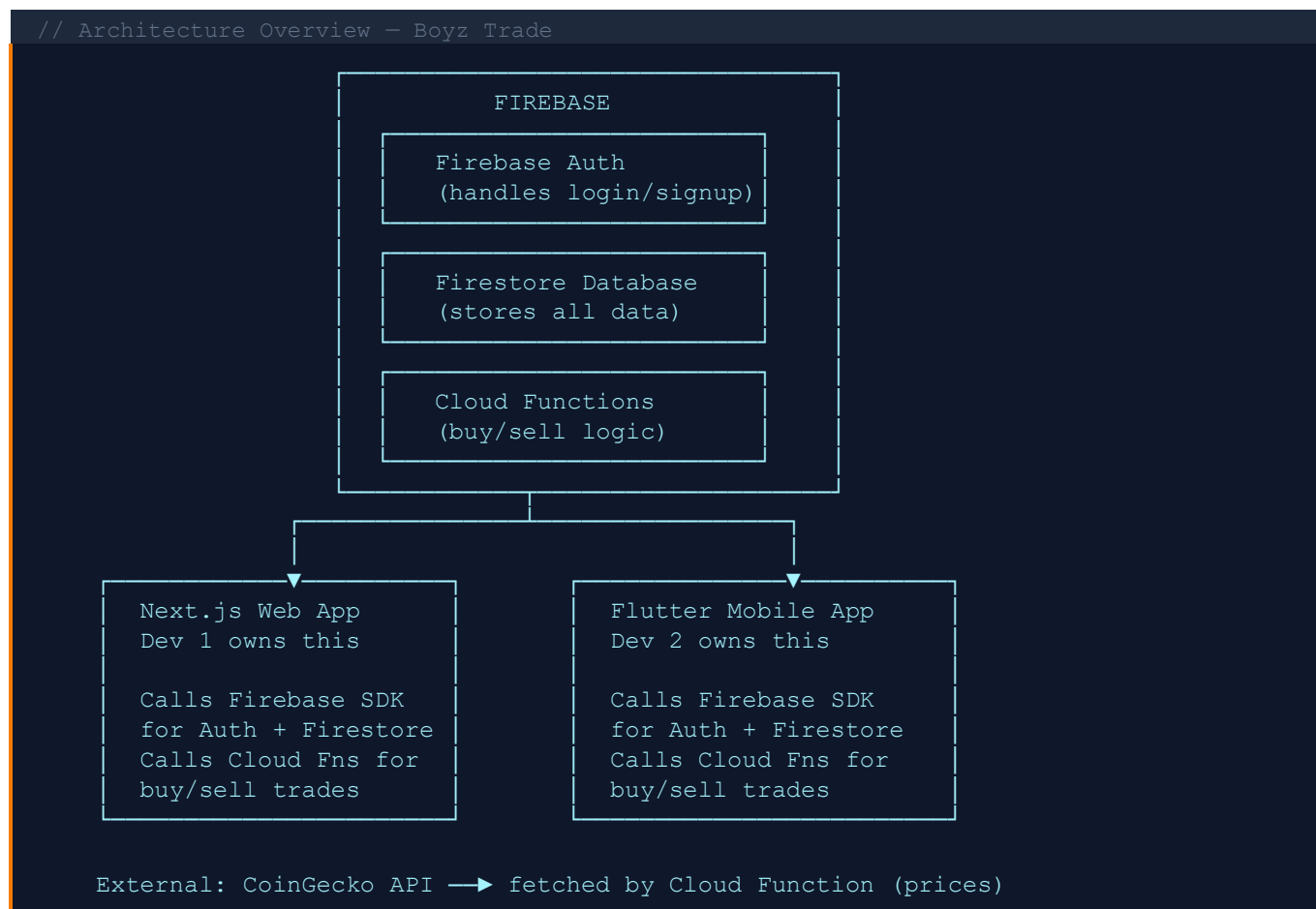
1.3 Target Users (Who Will Use This App?)

- Crypto beginners (16–40 years old) who want to learn trading
- Students interested in finance and technology
- Anyone who wants to practice without risking real money

02 System Architecture

How Next.js, Flutter, and Firebase connect together

Both Next.js and Flutter connect to the same Firebase backend. Neither frontend talks directly to the other — all data flows through Firebase.



📌 Important rule: Frontends never call CoinGecko directly

- Both Next.js and Flutter must get prices by calling the Cloud Function — not CoinGecko directly.
- Why: CoinGecko has rate limits. If 100 users open the app at once, 100 direct calls would get you blocked.
- The Cloud Function fetches the price once, caches it for 30 seconds, and serves all users from that cache.

03 Firebase Cloud Functions

Your backend — what functions to build and how they work

Cloud Functions are your backend. The frontend calls a function, the function reads/writes Firestore or fetches from CoinGecko, and returns the result. All buy/sell logic lives here.

⚠ The one thing Firebase requires careful attention

- Buy and Sell operations **MUST** use Firestore Transactions — this is the most critical rule in the whole project.
- A Firestore Transaction means: all steps happen together, or none of them happen.
- Example: if deducting the balance succeeds but adding to portfolio fails — the user loses money but gets nothing. That is a bug that destroys trust.
- Section 05 of this document explains exactly how to write this correctly with code examples.

3.1 List of All Cloud Functions

Function Name	Trigger	Auth	What It Does
<code>onUserCreated</code>	Auth trigger	Auto	Fires when a new user signs up. Creates their Firestore document with \$10,000 balance.
<code>getPrices</code>	HTTP call	Authenticated	Fetches live prices for 5 assets from CoinGecko. Returns cached result if fresh.
<code>executeBuy</code>	HTTP call	Authenticated	Runs buy trade logic inside a Firestore Transaction. CRITICAL — see Section 05.
<code>executeSell</code>	HTTP call	Authenticated	Runs sell trade logic inside a Firestore Transaction. CRITICAL — see Section 05.
<code>getPortfolio</code>	HTTP call	Authenticated	Returns user's holdings + balance + total P&L calculation.
<code>getTransactions</code>	HTTP call	Authenticated	Returns user's full trade history, sorted by most recent.

3.2 How Both Frontends Call a Cloud Function

Both Next.js and Flutter call Cloud Functions the same way — using the Firebase SDK. Here is the pattern for both:

```
// Next.js — calling a Cloud Function
// — Next.js (JavaScript) —————
import { getFunctions, httpsCallable } from 'firebase/functions';

const functions = getFunctions();
const executeBuy = httpsCallable(functions, 'executeBuy');
```

```
const result = await executeBuy({
  asset: 'BTC',
  usdAmount: 500
});

console.log(result.data); // { success: true, newBalance: 9500 }
```

```
// Flutter (Dart) — calling the same Cloud Function

// — Flutter (Dart) —————
final functions = FirebaseFunctions.instance;

final result = await functions
  .httpsCallable('executeBuy')
  .call({
    'asset': 'BTC',
    'usdAmount': 500,
  });

print(result.data); // {success: true, newBalance: 9500}
```

04 Firestore Data Structure

How and where all data is stored

Firestore is a NoSQL database. Data is organized into collections (like tables) and documents (like rows). Here is the full data structure for Boyz Trade.

 Collection: users / {userId}			
Field	Type	Default	Description
uid	string	from Auth	Firebase Auth user ID — same as the document ID
email	string	from Auth	User's email address
balance	number	10000	Current virtual USD balance. NEVER goes below 0.
createdAt	timestamp	now()	When the account was created

 Collection: users / {userId} / holdings / {assetSymbol}			
Field	Type	Default	Description
symbol	string	e.g. 'BTC'	Asset symbol. Also used as the document ID.
quantity	number	0	How many units the user owns. e.g. 0.015 BTC
avgBuyPrice	number	0	Average price paid per unit. Used to calculate P&L.
updatedAt	timestamp	now()	Last time this holding was updated

 Collection: transactions / {transactionId}			
Field	Type	Default	Description
userId	string	from Auth	The user who made this trade
type	string	'BUY' 'SELL'	Only these two values are allowed
asset	string	e.g. 'BTC'	Which asset was traded
quantity	number	calculated	How many units were bought or sold
priceAtTrade	number	live price	The exact price at the moment of the trade
totalUsd	number	calculated	Total USD value = quantity × priceAtTrade
createdAt	timestamp	now()	Exact time of the trade

 Collection: `priceCache` / `'latest'` (single document)

Field	Type	Default	Description
BTC	<code>number</code>	<code>live</code>	Current BTC price in USD
ETH	<code>number</code>	<code>live</code>	Current ETH price in USD
SOL	<code>number</code>	<code>live</code>	Current SOL price in USD
DOGE	<code>number</code>	<code>live</code>	Current DOGE price in USD
ADA	<code>number</code>	<code>live</code>	Current ADA price in USD
<code>fetcheAt</code>	<code>timestamp</code>	<code>now()</code>	When prices were last fetched. Used to decide if cache is still fresh.

05 CRITICAL: Buy & Sell Logic with Firestore Transactions

The most important section — read this carefully before writing any code

Why this section is marked CRITICAL

- Buy and Sell logic touches money (even virtual money). If done wrong, a user could lose balance without getting assets, or get assets without losing balance.
- This is the vending machine problem: either the machine gives you the snack AND takes your coins, or it does neither. It must NEVER take your coins and give nothing.
- Firestore Transactions solve this — all steps run together or all fail together. There is no in-between state.
- This logic must live in Cloud Functions ONLY — never in the frontend. Frontends can be tampered with.

5.1 Buy Logic — Full Step-by-Step

Step	What Happens	What to Check / Do
1	Receive request	Get: userId, asset (e.g. 'BTC'), usdAmount (e.g. 500)
2	Get current price	Fetch from priceCache Firestore doc. Must be fresh (under 30 sec).
3	Calculate quantity	quantity = usdAmount / currentPrice. e.g. 500 / 50000 = 0.01 BTC
4	START transaction	Open Firestore Transaction — all steps below are inside it.
5	Read user balance	Read users/{userId}.balance inside transaction.
6	Check balance	If balance < usdAmount → throw error 'Insufficient balance'. Stop here.
7	Deduct balance	New balance = balance - usdAmount. Write to users/{userId}.balance
8	Update holdings	If holding exists: recalculate avgBuyPrice and add quantity. If new: create holding document.
9	Log transaction	Write new document to transactions collection.
10	COMMIT transaction	Firestore saves all changes atomically. If any step failed, ALL are rolled back.
11	Return result	Send back: { success: true, newBalance, quantity, asset }

```
// Cloud Function — executeBuy (JavaScript)
// — executeBuy Cloud Function —————
const { onCall } = require('firebase-functions/v2/https');
const { getFirestore, FieldValue } = require('firebase-admin/firestore');

exports.executeBuy = onCall(async (request) => {
  const { asset, usdAmount } = request.data;
  const userId = request.auth.uid; // Firebase verifies this automatically

  if (!userId) throw new Error('Not authenticated');
  if (!asset || !usdAmount || usdAmount <= 0) throw new Error('Invalid input');
```



```

const db = getFirestore();

// Step 1: Get cached price (must be fresh)
const cacheDoc = await db.collection('priceCache').doc('latest').get();
const cache = cacheDoc.data();
const ageSeconds = (Date.now() - cache.fetchedAt.toMillis()) / 1000;

if (ageSeconds > 60) throw new Error('Price data is stale. Try again.');
```

```

const currentPrice = cache[asset];
if (!currentPrice) throw new Error('Asset not supported');
```

```

const quantity = usdAmount / currentPrice;
```

```

// Step 2: Run Firestore Transaction (ALL or NOTHING)
const result = await db.runTransaction(async (tx) => {

  // Read user document inside transaction
  const userRef = db.collection('users').doc(userId);
  const userDoc = await tx.get(userRef);
  const balance = userDoc.data().balance;

  // Check balance BEFORE doing anything
  if (balance < usdAmount) {
    throw new Error('Insufficient balance');
  }

  // Deduct balance
  tx.update(userRef, { balance: balance - usdAmount });

  // Update or create holding
  const holdingRef = db.collection('users').doc(userId)
    .collection('holdings').doc(asset);
  const holdingDoc = await tx.get(holdingRef);

  if (holdingDoc.exists) {
    const existing = holdingDoc.data();
    const totalQty = existing.quantity + quantity;
    const newAvgPrice = ((existing.quantity * existing.avgBuyPrice) +
      (quantity * currentPrice)) / totalQty;
    tx.update(holdingRef, {
      quantity: totalQty,
      avgBuyPrice: newAvgPrice,
      updatedAt: FieldValue.serverTimestamp()
    });
  } else {
    tx.set(holdingRef, {
      symbol: asset,
      quantity,
      avgBuyPrice: currentPrice,
      updatedAt: FieldValue.serverTimestamp()
    });
  }
});

```

```
// Log the transaction
const txRef = db.collection('transactions').doc();
tx.set(txRef, {
  userId, type: 'BUY', asset, quantity,
  priceAtTrade: currentPrice,
  totalUsd: usdAmount,
  createdAt: FieldValue.serverTimestamp()
});

return { newBalance: balance - usdAmount };
});

return { success: true, newBalance: result.newBalance, quantity, asset };
});
```

5.2 Sell Logic — Full Step-by-Step

Step	What Happens	What to Check / Do
1	Receive request	Get: userId, asset (e.g. 'BTC'), quantity (e.g. 0.005)
2	Get current price	Fetch from priceCache. Must be fresh.
3	Calculate USD return	usdReturn = quantity × currentPrice. e.g. 0.005 × 55000 = \$275
4	START transaction	Open Firestore Transaction.
5	Read holding	Read users/{userId}/holdings/{asset} inside transaction.
6	Check quantity owned	If holding does not exist OR holding.quantity < requested quantity → throw error 'Insufficient holdings'.
7	Reduce holding	New quantity = holding.quantity - requested quantity. If new quantity = 0, delete the document.
8	Add to balance	Read balance, add usdReturn. Write new balance.
9	Log transaction	Write to transactions collection with type: 'SELL'.
10	COMMIT transaction	Firestore saves all changes atomically.
11	Return result	Send back: { success: true, newBalance, usdReturn }

06 Feature Specifications

Every screen and feature — what it does, inputs, outputs, rules



FEATURE GROUP A — Authentication

F-01 User Registration

Feature ID	F-01
User Story	As a new user, I want to sign up so I can start virtual trading.
Input	Email + Password
Firebase Flow	1. Call Firebase Auth createUserWithEmailAndPassword() 2. This triggers onUserCreated Cloud Function automatically 3. Cloud Function creates users/{uid} document with balance: 10000
Output	Account created. User logged in. Balance set to \$10,000.
Rules	Email must be valid. Password min 8 characters. Duplicate email → Firebase returns clear error automatically.
Priority	Must Have

F-02 User Login

Feature ID	F-02
User Story	As a returning user, I want to log in to see my portfolio.
Input	Email + Password
Firebase Flow	Call Firebase Auth signInWithEmailAndPassword(). Firebase returns user object + ID token automatically.
Output	User is authenticated. Firebase SDK stores session. Redirect to dashboard.
Rules	Wrong credentials → Firebase returns AUTH/WRONG_PASSWORD error. Show friendly message. Session persists until user logs out.
Priority	Must Have



FEATURE GROUP B — Market Prices

F-03 View Live Prices

Feature ID	F-03
------------	------

 Boyz Trade — PRD v1.0 MVP Next.js + Flutter + Firebase	
User Story	As a user, I want to see live crypto prices so I can decide when to trade.
Assets	BTC, ETH, SOL, DOGE, ADA (5 assets maximum for MVP)
Flow	Frontend calls getPrices Cloud Function → Function checks priceCache in Firestore → If cache is older than 30 seconds: fetch fresh from CoinGecko, update cache → Return prices to frontend
Display	Asset name, symbol, current USD price, 24h price change % (green = up, red = down)
Refresh	Auto-refresh every 30 seconds. Manual pull-to-refresh also supported.
Edge Case	CoinGecko down → return last cached price with visible warning: 'Prices may be delayed'.
Priority	Must Have

⚡ FEATURE GROUP C — Trading

F-04 Buy Asset

Feature ID	F-04
User Story	As a user, I want to buy a crypto asset using my virtual balance.
Input	Select asset + enter USD amount to spend
Flow	Frontend calls executeBuy Cloud Function → Function runs Firestore Transaction (see Section 05) → Returns updated balance and quantity bought
Output	Balance reduced. Holding added or increased. Transaction logged. UI updates.
Rules	Minimum trade: \$1 USD. Cannot buy more than current balance. Balance cannot go below \$0.
Edge Cases	Insufficient balance → show error with current balance. Amount = 0 → prevent form submission. Price stale → Cloud Function returns error.
Priority	Must Have — CORE FEATURE

F-05 Sell Asset

Feature ID	F-05
User Story	As a user, I want to sell a crypto asset I own to get USD back.
Input	Select asset from holdings + enter quantity to sell (or tap 'Sell All')
Flow	Frontend calls executeSell Cloud Function → Function runs Firestore Transaction → Returns updated balance and USD received
Output	Holding reduced or removed. Balance increased. Transaction logged. UI updates.

Rules	Cannot sell more than owned. Partial sells allowed. If quantity after sell = 0, remove holding document.
Edge Cases	Asset not in portfolio → prevent sell button from appearing. Sell quantity > owned quantity → show error with owned amount.
Priority	Must Have — CORE FEATURE



FEATURE GROUP D — Portfolio

F-06 Portfolio Dashboard

Feature ID	F-06
User Story	As a user, I want to see all my holdings and whether I am in profit or loss.
Flow	Call getPortfolio Cloud Function → Function reads holdings subcollection + balance + fetches current prices → Calculates P&L for each asset → Returns full portfolio data
Display	Cash balance, each holding (name + quantity + avg buy price + current price + P&L %), total portfolio value, total overall P&L from \$10,000 starting amount
Color Coding	Green = profit, Red = loss. Applied to both individual assets and overall P&L.
P&L Formula	$P\&L \text{ per asset} = (\text{currentPrice} - \text{avgBuyPrice}) \times \text{quantity}$. Total P&L = total portfolio value - 10000.
Priority	Must Have

F-07 Transaction History

Feature ID	F-07
User Story	As a user, I want to see a history of all my trades.
Flow	Call getTransactions Cloud Function → Function queries transactions collection where userId = current user, ordered by createdAt descending
Display	Date/time, asset, type (BUY or SELL), quantity, price at trade, total USD
Priority	Should Have

07 Development Roadmap

Step-by-step — in the correct order. Do not skip steps.

Phase	Name	Tasks
Phase 1	Firestore Setup	- Creates Firestore project on console.firebase.google.com - Enable Firestore, Firebase Auth (email/password), and Cloud Functions - Install Firestore Emulator Suite for local testing
Phase 2	Cloud Functions	- Write onUserCreated — auto-create user document on signup - Write getPrices — fetch from CoinGecko, cache in Firestore - Write executeBuy — with full Firestore Transaction (Section 05) - Write executeSell — with full Firestore Transaction (Section 05) - Write getPortfolio — read holdings + calculate P&L - Write getTransactions — return trade history - Test ALL functions using Firestore Emulator
Phase 3	Firestore Rules	- Write Firestore Security Rules — users can only read/write their own data - Users collection: only owner can read/write their document - Holdings subcollection: only owner can read/write - Transactions: only owner can read, Cloud Functions write only - priceCache: everyone can read, only Cloud Functions can write - Test rules using Firestore Rules Playground in console
Phase 4	Next.js Web	- Install Firestore SDK: npm install firebase - Build Firestore config file (initializeApp) - Build Login screen + Register screen using Firebase Auth - Build Home screen — call getPrices, display price list with auto-refresh - Build Trade screen — buy/sell form calling executeBuy/executeSell - Build Portfolio screen — call getPortfolio, display holdings + P&L - Build Transactions screen — call getTransactions
Phase 5	Flutter Mobile	- Add firebase_core, firebase_auth, cloud_firestore, cloud_functions packages - Set up google-services.json (Android) and GoogleService-Info.plist (iOS) - Build same 4 screens as web — same Cloud Function calls, different UI - Test on Android emulator + iOS simulator - Handle mobile-specific UX: keyboard padding, touch targets, loading states
Phase 6	Testing	- Test full buy flow end-to-end: signup → buy → check portfolio - Test full sell flow: sell partial → sell all → check holdings removed - Test edge cases: buy with 0 balance, sell more than owned, stale prices - Test both platforms call the same Cloud Functions and get same results - Fix all bugs before deployment
Phase 7	Deploy	- Deploy Cloud Functions: firebase deploy --only functions - Deploy Firestore Rules: firebase deploy --only firestore:rules - Deploy Next.js web: firebase deploy --only hosting - Flutter: build APK for Android testing (flutter build apk) - Share with 3–5 test users and collect feedback